APPENDIX

A. Technical Implementation of Pos-to-Pos Non-Collision Module

Due to differences in action space and limitations in the precision of the retargeting algorithm, the configuration of a dexterous robotic hand often generates invalid self-collision configurations. These invalid configurations not only lack operational utility but also risk system failure or hardware damage. To address this issue, we propose a method for mapping invalid configurations to their closest valid counterparts, enabling recovery from self-collision scenarios.

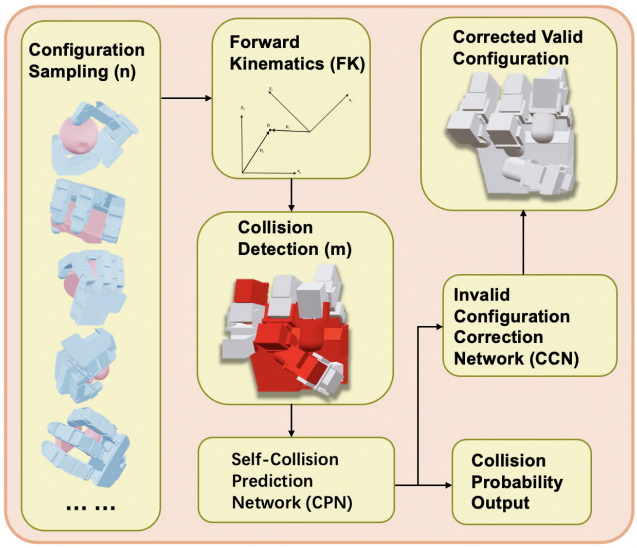


Fig. 8: Pipeline of the Non-collision Module

Fig. 8 illustrates our pos2pos implementation pipeline, which consists of several interconnected components for handling robot hand configurations.

1) Self-Collision Prediction Network (CPN): To facilitate the transformation from invalid to valid configurations, we first develop a **Self-Collision Prediction Network** (CPN). The primary objective of CPN is to predict the likelihood of self-collision for each link within a given joint configuration.

The training dataset is generated by uniformly sampling n configurations from the robot's action space. For each sampled configuration, the system employs forward kinematics (FK) to compute the robot's pose. A collision detection algorithm (e.g., geometric or physics-based) then checks the pose to derive m collision labels for the links. Each label indicates whether a link is in a collision state.

The CPN takes joint configurations as input and outputs collision probabilities for all joints. We train the network using the binary cross-entropy (BCE) loss function, defined as:

$$L_{\text{CPN}} = \frac{1}{m}\sum_{i=1}^{m}\text{BCE}(p_i, t_i), \tag{5}$$

where p_i and t_i represent the predicted and true collision probabilities, respectively.

2) Invalid Configuration Correction Network (CCN): Building on the CPN, we introduce an **Invalid Configuration Correction Network** (CCN) to map invalid configurations to valid ones. The CCN takes an invalid configuration as input and outputs a corrected configuration that minimizes collision risks while closely resembling the original input.

The CCN training process minimizes a composite loss function comprising two components:

- **Mean Squared Error (MSE) Loss**: Ensures the corrected configuration closely resembles the original configuration.

$$L_{\text{MSE}} = \frac{1}{n}\sum_{i=1}^{n}(\hat{q}_i - q_i)^2, \tag{6}$$

  where q_i and $\hat{q}_i$ denote the original and corrected joint configurations, respectively.
- **Collision Probability Loss**: Leverages the CPN to compute the mean collision probability of the corrected configuration and aims to minimize this value.

$$L_{\text{Collision}} = \frac{1}{m}\sum_{i=1}^{m} p_i(\hat{q}), \tag{7}$$

  where $p_i(\hat{q})$ represents the collision probability of joint i in the corrected configuration $\hat{q}$.

We define the total loss function as:

$$L = \alpha L_{\text{MSE}} + \beta L_{\text{Collision}}, \tag{8}$$

where α and β are hyperparameters balancing the two loss components.

3) Explanation and Optimization Strategy: The loss terms in the proposed framework serve distinct roles:

- L_{MSE} ensures the corrected configuration retains continuity with the original input.
- $L_{\text{Collision}}$ minimizes the likelihood of self-collision in the corrected configuration.
- The hyperparameters α and β significantly influence the training outcomes, and we optimize their values through grid search.

The CCN employs a fully connected multi-layer perceptron (MLP) architecture. The input is the invalid joint configuration q, and the output is the corrected configuration $\hat{q}$. We train the model using the Adam optimizer with a learning rate η and monitor convergence via the collision rate on a validation dataset.

4) Summary: By integrating the CPN and CCN, we efficiently transform invalid self-collision configurations into valid ones. This approach ensures the validity and continuity of robotic configurations, laying a robust foundation for subsequent task execution.

B. Bill of Materials (BOM)

Our teleoperation system supports a modular architecture with flexible input configurations. **Not all devices shown in Fig. 2 are required simultaneously.** Instead, the system requires **one device from each of the two input groups** on the left side of the diagram:

- **Wrist pose acquisition:** RGB(-D) camera, IMU mocap suit, or similar.
- **Hand gesture acquisition:** Mocap gloves, AR tracking, or EMG-based sensing.

This design allows users to build their system using available hardware, optimizing for cost, performance, or ease of use. For example, while a VR headset can serve as a unified sensor in both categories, it is not necessary. Our system can be operated using a standard external monitor, as done in our experiments.

Experimental Setup Used in This Paper: In our experiments, we selected a configuration based on performance, generalizability, and affordability:

- **Mocap Gloves:** $500
  - Kickstarter Product Page
- **IMU-Based Motion Capture Suit:** $200
  - Rebocap Product Page
- **RGB-D Camera (Intel RealSense D435):** $300
  - Amazon Product Page
- **Total Cost:** Approximately **$1000**

Note: Since our method does not depend on depth data, the RGB-D camera can be replaced by a lower-cost RGB camera, further reducing the overall system cost.

Remarks on Reproducibility and Flexibility: The modular nature of the TelePreview architecture allows for hardware substitution based on availability or task requirements. We provide this BOM to facilitate future replication efforts and to emphasize that **our system's affordability claim refers to the teleoperation input interface only**, not the robot arm or end-effector hardware.

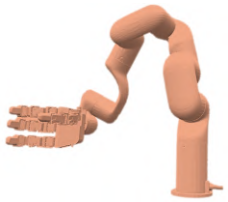
C. Preview Rendering and Multi-view Display Details

1) Preview Rendering Implementation: We use the pyrender library to render the virtual robot through the float camera. The preview image is composited with the real camera feed using alpha blending, creating an overlay where the virtual robot appears aligned with the physical environment. This requires calibration of virtual camera parameters to match the physical cameras, ensuring the preview accurately reflects the robot's intended configuration.

2) Multi-view Display System: Our system supports multiple viewpoints to aid spatial reasoning and trajectory checking. We deploy third-person and top-down cameras, rendered and shown simultaneously on an external monitor. The system does not require a VR headset. For instance, during manipulation tasks, users can simultaneously monitor the robot's vertical position from the third-person view while checking its planar alignment from the top-down view.

D. Reproduction Experience of Baseline Methods

We tested several typical vision-based teleoperation methods under our experimental setup: OpenTeach, OpenTelevision, and AnyTeleop. For methods that did not support LeapHand in the open-source code, we implemented the corresponding parts ourselves to apply these methods.



(a) Preview Rendering

(b) Third-person View

(c) Top-down View

Fig. 9: Rendering and Multi-view Visualization System.

OpenTeach: Following the guidelines in the open-source repository for adding new hardware, we implemented the retargeting module for LeapHand. However, we encountered two significant issues. First, gesture recognition based on visual input is highly sensitive to occlusions. When the back of the hand is fully occluded or the side of the hand is partially blocked, the positioning of the fingers can deviate substantially—particularly with the Meta Quest 3, which has less accurate hand tracking compared to the Apple Vision Pro. Second, the visual approach depends heavily on precise hand calibration, requiring careful adjustment of parameters to achieve acceptable retargeting results. Together, these two factors led to instability in the accuracy of teleoperation in some instances, ultimately impacting the success rate of task completion.

OpenTelevision: The open-source code only provides the necessary components for the 6-DoF Inspire Hand, with no guidance on how to integrate new dexterous hands. Additionally, many hand-specific hyperparameters lack proper documentation. After attempting to integrate LeapHand using the dex-retargetting code and redoing the joint mapping, we were able to align the retargeting results with the hand's movements. However, due to LeapHand's significantly higher degrees of freedom, we encountered severe self-collision issues. During deployment, motor collisions occurred, rendering effective teleoperation impossible. Such self-collision problems did not arise with the original Inspire Hand, as its 6-DoF design inherently prevents the possibility of self-collisions.

AnyTeleop: The retargeting code used here is also based on dex-retargetting. For LeapHand, a high-DOF dexterous hand with a mechanical design that has a higher likelihood of self-collision, relying solely on fingertip-based inverse kinematics (IK) resulted in frequent self-collisions. This posed significant issues during teleoperation.

E. User Study Methodology and Subjective Evaluation

To complement the quantitative performance metrics presented in the main paper, we conducted a user study to eval-

uate the subjective experience and learning curve associated with our system. The study focused on two primary aspects: the perceived workload required to complete tasks and the practice time needed for new users to gain confidence.

1) Subjective Workload Assessment using NASA-TLX: To measure subjective workload, we employed the NASA Task Load Index (NASA-TLX), a well-established and multidimensional assessment tool developed by NASA's Ames Research Center. It is widely used in human-computer interaction and ergonomics to assess the perceived workload of a task. After completing all five tasks under both the 'with Preview' and 'without Preview' conditions, each participant completed the NASA-TLX questionnaire.

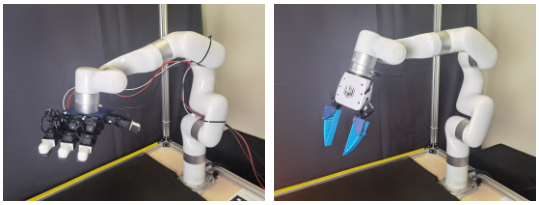
The NASA-TLX procedure consists of two parts. First, participants rate their experience on a 20-point scale across the following six subscales:

- **Mental Demand:** How much mental and perceptual activity was required (e.g., thinking, deciding, calculating, remembering, looking)? Was the task easy or demanding, simple or complex?
- **Physical Demand:** How much physical activity was required (e.g., pushing, pulling, turning, controlling)? Was the task easy or demanding, slow or brisk?
- **Temporal Demand:** How much time pressure did you feel due to the rate or pace at which the tasks or task elements occurred? Was the pace slow and leisurely or rapid and frantic?
- **Performance:** How successful do you think you were in accomplishing the goals of the task set by the experimenter? How satisfied were you with your performance?
- **Effort:** How hard did you have to work (mentally and physically) to accomplish your level of performance?
- **Frustration:** How insecure, discouraged, irritated, stressed, and annoyed versus secure, gratified, content, relaxed, and complacent did you feel during the task?

In the second part, participants complete a series of pairwise comparisons of these six dimensions to create a weighted score that reflects the individual's definition of workload for that specific task. The final NASA-TLX score is a weighted average of the ratings, providing a comprehensive and personalized measure of perceived workload. Our results indicated a significantly lower overall NASA-TLX score for the **'with Preview'** condition, confirming that our system not only improves objective performance but also reduces the cognitive and physical strain on the operator.

2) Practice Time to Confidence: We also recorded the time participants spent practicing with each control modality before they reported feeling comfortable enough to begin the formal trials. We compared three conditions: our proposed method **w/ Preview**, the same setup **w/o Preview**, and a baseline **vision-based method** [3]. The results, shown in the main paper, demonstrate that participants learned to use our full TelePreview system most quickly, reaching task readiness in a shorter time and with less variability across users compared to the other modalities. This supports the claim that our system is more intuitive and easier for new users to master.

F. End-Effector Integration Details



(a) LEAP Hand (b) Gripper

Fig. 10: Deployment on Different Robots.

To demonstrate the generalizability of our system, we deployed TelePreview on a Ufactory xArm robot equipped with three different end-effectors(See in Fig. 10):

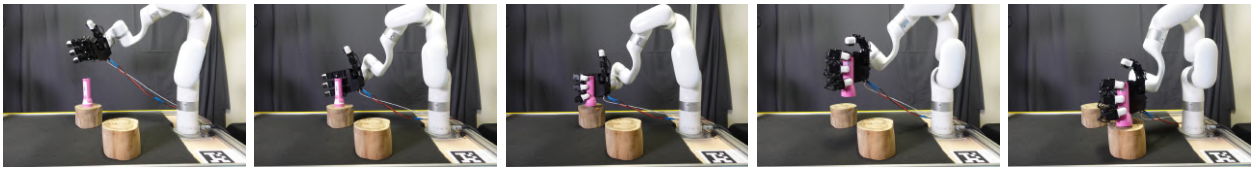
- **LeapHand (multi-DoF anthropomorphic hand)**: Controlled via direct mapping of 16 captured joint angles from the mocap glove through our SMPL-X based retargeting pipeline. The preview and execution follow the full configuration space of the robot hand, enabling rich dexterous behaviors.
- **Parallel-jaw Gripper (single-DoF)**: We selected a representative finger joint angle from the mocap glove and used its value to control the gripper's open/close motion. This allowed the system to retain the preview feature without the need for full-hand mapping.
- **Vacuum Gripper (binary actuator)**: We applied a threshold to the same glove joint signal to produce a binary on/off activation, simulating "grasp" or "release" behavior. This minimal control model was still compatible with the preview system.

As shown in Table III, the preview system yields the greatest execution time improvement with the LeapHand, supporting our claim that TelePreview is most beneficial for high-DoF, complex manipulation tasks.

G. Task Descriptions and Success Criteria

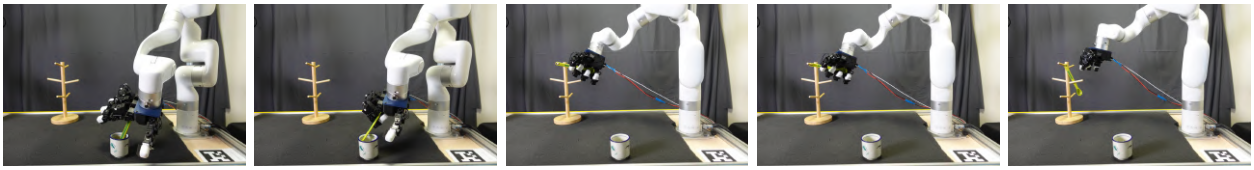
We evaluate our system on five challenging manipulation tasks (Fig. 11-15). A trial is considered successful if the following conditions are met:

- **Pick & Place:** A flashlight is grasped, transported to a target zone, and released in a stable, upright position.
- **Hang:** The small hole on the handle of a spoon is successfully inserted onto a narrow peg. This is a highly challenging task requiring precise six-dimensional control of the end-effector.
- **Pour:** At least 90% of the beans from one container are poured into a target container without spilling.
- **Box Rotation:** A box is grasped, lifted, rotated in-hand by approximately 90 degrees, and placed back down stably.
- **Cup Stacking:** A second cup is placed onto a base cup, forming a stable stack that remains standing after the robot retracts.



(a) (b) (c) (d) (e)

Fig. 11: Pick&Place Visualization

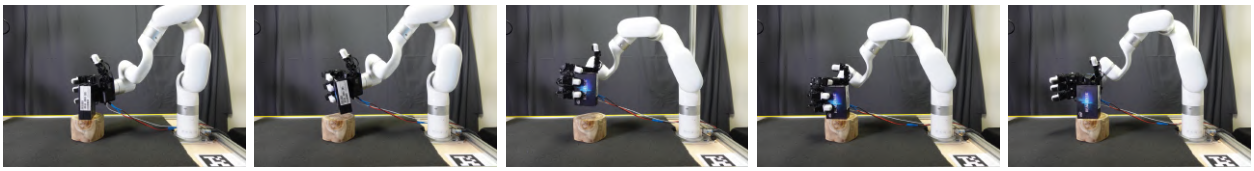


(a) (b) (c) (d) (e)

Fig. 12: Hang Visualization

(a) (b) (c) (d) (e)

Fig. 13: Pour Visualization



(a) (b) (c) (d) (e)

Fig. 14: Box Rotation Visualization

(a) (b) (c) (d) (e)

Fig. 15: Cupstack Visualization